



**Numerical Modeling of Nonlinear Systems:
A Case Study of the Lorenz System**

Department of Mathematics

Faculty Of Science

Thesis project

2024-2025

Supervised By

Dr/ Salma Shatta



التعامل العددي مع الأنظمة غير الخطية:

دراسة حالة لنظام لورينز

مشروع تخرج بتاريخ

2025-2024

قسم الرياضيات

كلية العلوم

تحت إشراف

د/ سلمى شطا

Student's Names and Approval Sheet

Name of Student	ID	Marks
Hassan Mohamed Hassan	352151231	
Karim Emad Fawzy	352151234	
Mostafa Osama Mostafa	352151183	

Approving Committee's Names

Name	Signature
Dr. Doha Muhammad	
Dr. Eman Ibrahim	
Dr. Salma A. Shatta	

Abstract

The Lorenz system, a Dynamical System that is a set of three coupled nonlinear differential equations, In this thesis we're going to study what's Lorenz System, What's Numerical Analysis and how to solve Lorenz System Numerically using Python.

Additionally, we will discuss how to analyze the stability of a system and to find the equilibrium points.

And in the final chapter we will discuss the application of Machine Learning on our dynamical system by classifying the output states of our system either to be chaotic or non-chaotic.

Abbreviation

Abbreviations	Meaning
ODE	Ordinary Differential Equation
DE	Differential Equation
RK	Runge-Kutta
ML	Machine Learning

Contents

Summary.....	ix
Chapter 1: Introduction	
1.1 Ordinary Differential Equations.....	5
1.2 Dynamical System and Mathematical Modeling.....	8
1.3 Python Software.....	10
Chapter 2: Numerical Methods for ODEs	
2.1 Numerical Methods vs. Analytical Methods.....	20
2.2 Types of Numerical Methods.....	20
Chapter 3: Lorenz System	
3.1 Solving the system to find equilibrium points.....	26
3.2 Solving the System of Differential Equations using Python.....	27
3.3 Stability Analysis.....	28
Chapter 4: Machine Learning Applications on Dynamical Systems	
4.1 Introduction To Machine Learning.....	32
4.2 Preprocessing The Data.....	33
4.3 Results And Analysis.....	36
References.....	40

List of Tables

1.1.4	Classification of ODEs.....	10
2.3.1	Adam Bashforth 1,2,3 and 4 coefficients and order of error.....	27
3.3.1	Classification of eigenvalues and stability.....	33

List of Figures

1.3-1	writing a code visualizing a direct application of mathematical modeling using a model called logistic population growth model.....	17
1.3-2	A code using NumPy library to manipulate matrices like adding them or using multiplication.....	21
1.3-3	using SciPy library to find the value of x of an equation.....	23
2.3-1	We established a function that returns the DE so that we can use it inside the Euler method function, Euler method here is entered inside an iteration until the upper bound in the interval is the point we're on.....	28
2.3-2	Applying the same problem mentioned in this section using RK4.....	29
3.2-1	Solving the Lorenz System with a time span with RK-modified.....	31
3.2-2	Showing the results of solving the Lorenz System Using RK-modified while using chaotic parameters.....	32
4.1-2	Explaining how classification algorithms work in the data points plane.....	37
4.1-1	visualization of gradient descent presenting how iteration led to minimum cost.....	36
4.2-1	Representing the tabular form of the data which shows 3 features and 1 output in 200 records.....	37
4.2-2	Python code that applies an encoder method to transform categories into numerical categories.....	38
4.2-3	The output of applying the OneHotEncoder on the dataset.....	38
4.2-4	Showing the code of applying the random forest classifier on the training set of our data.....	39
4.3-1	running the accuracy function to see how well the model will get if we apply it to unknown values.....	39

Summary

The main purpose of this thesis is to understand dynamical system, Specifically Lorenz System and study the behaviors of the system identifying how a chaotic model behave and how to solve the model numerically with experimenting if Statistical models connected to Machine Learning can be applicable in our field or not.

This Thesis is Organized as follows:

- **Chapter 1**

This chapter introduces us to what's ordinary differential equations, How It can be solved and introduces us to dynamical systems, Mathematical Modeling and stability analysis with knowledge of the uses of python and its libraries.

- **Chapter 2**

This chapter presents the what's Numerical Methods, the difference between analytical solutions vs. Numerical solutions and provides some important Numerical Methods.

- **Chapter 3**

We aim in this chapter to explain Lorenz system of equations, solving the model numerically by python and analyzing the stability of the models' equilibrium points.

- **Chapter 4**

This chapter presents the definition of ML, what is Classification algorithm, Applying the algorithm to our model, And analysis the results.

Chapter 1

Introduction

1.1 Ordinary differential equations

Differential equations have been the primary mathematical tool that physicists have relied on to model physical phenomena for a long time, and this is likely to continue. However, modeling other types of phenomena can be quite challenging but also very useful, especially when there are no physical principles or governing equations to guide the modeling process. Examples of such phenomena include biological systems, economics, and more. We start with the essential definitions we find in to emphasize the problems' structure, its conditions for solving it and we study its stability, another important topic we need to specify the classification of the problems. Finally, we study their existence and uniqueness, which revisited again in fractional order problem also.

1.1.1 Definitions

Starting with formal definition of ODEs, initial value, its solution through Definition 1 to 3 and Example 1.1.1 to 1.1.3

Definition 1. [2] *An ordinary differential equation* is an equation which relates unknown function and some of its derivatives, here the implicit form

$$\Phi(y^{(n)}, y^{(n-1)}, \dots, y', y, x) = 0$$

Where $x \in I$ is the independent variable, $I \subseteq \mathbb{R}$ is the domain of the equation, and all of $y, y', \dots, y^{(n)}$ are functions of x

In Definition 1, the equation is in the Implicit form

Example 1.1.1. Examples of differential equations

- | | |
|----------------------|-------------------------|
| 1. $y' = cy$ | 3. $y'' = c^2y$ |
| 2. $(y^2) + x^2 = 0$ | 4. $y'(y - y_o) = ce^x$ |

Definition 2 (IVP). A given system is called initial value problem. If there exist a differential equation (or a system) with subsidiary conditions all are given values of the unknown function and a regarded number of its derivatives (less than the order by only once) for a fixed point t_0

$$\begin{aligned}\Phi(y^{(n)}, y^{(n-1)}, \dots, y', y, x) &= 0 \\ \{y(t_0) = y_0, y'(t_0) = y'_0, \dots, y^{(n-1)}(t_0) = y_0^{(n-1)}\}\end{aligned}$$

For simplicity the initial value is a point given from the solution (the expected solution of the problem, data value or some measurement). Initial value might destroy the whole problem if it breaks the existence or the uniqueness of the solution as we will see in subsection 1.1.3

Example 1.1.2. An example of IVP problem

$$\begin{aligned}y'' &= 4y \\ y(0) &= -2, \quad y'(0) = 2\end{aligned}$$

Definition 3. A function “f” is called a solution if it satisfies the problem equation and its initial conditions.

$$\begin{aligned}\Phi(y^{(n)}, y^{(n-1)}, \dots, y', y, x) &= \Psi(f(t)) \\ \{f(t_0) = y_0, f'(t_0) = y'_0, \dots, f^{(n-1)}(t_0) = y_0^{(n-1)}\}\end{aligned}$$

Example 1.1.3. The function $y(x) = 4e^{3x}$ is a solution for the following IVP

$$\begin{aligned}y' &= 3y \\ y(0) &= 4\end{aligned}$$

1.1.2 Classification of ODEs

Most applications of ODEs such as physical phenomenon, biological, ...etc, doedon'tange by time, which give us a quiet useful class of ODEs to focus on its properties and its physical intuition.

Definition 4 (Autonomous System). An *IVP* is called **autonomous** if the differential equation has no appearance of the time at all, as follows:

$$\begin{aligned}\dot{x} &= f(x), \quad \dot{y} = g(y) \\ x(0) &= x_0, \quad y(0) = y_0\end{aligned}$$

Definition 5 (The order). : The order of an ODE is the highest derivative order in whole equation

$$y^{(n)} = \Phi(y^{(n-1)}, y^{(n-2)}, \dots, y', y, x)$$

Another classified property which has wide attention and importance is order, order represents a physical quantity as (rate of change- velocity or acceleration), note that velocity isn't only for moving bodies but can represent concentrations, densities, population rise,...etc. the following two classes of ODEs (Linearity- Homogeneity) are more abstract concept for some special cases (Waves- Linear growth) make the problem easier to solve and well explained.

Definition 6 (Linearity): A differential equation is called Linear if all the function and its derivatives are a first-degree power if it exists with coefficient is a function of the independent variable

$$A_0 y^{(n)} + A_1 y^{(n-1)} + \dots + A_{n-1} y' + A_n y = f(t)$$

While $A_i(t)$, $i = 1, 2, \dots, n$

Definition 7 (Homogeneity): A differential equation is homogeneous if it is linear and every term in the equation has an unknown function or one of its derivatives, otherwise it isn't. Also, we say that a linear equation is homogeneous if the free term is equal to zero.

$$A_0 y^{(n)} + A_1 y^{(n-1)} + \dots + A_{n-1} y' + A_n y = 0$$

Here are some examples of classification of some ODEs to make those definitions clear as possible

Example 1.1.4. Some Classification of ODE's

1.	$y' = cy$	1 st order	Linear	Homogeneous
2.	$(y')^2 + x^2 = 0$	1 st order	Non-Linear	Non-Homo
3.	$y'' = c^2 y$	2 nd order	Linear	Homogeneous
4.	$y'(y - y_0) = ce^x$	1 st order	Non-Linear	Non-Homo

1.1.2 Existence & uniqueness

Existence and uniqueness are important concepts for applied mathematics, especially modeling process which necessary to show that the model has solution and it's unique. Also, the importance of existence and uniqueness is showed up for numerical approximation for assuming high level of non-linearity to make best fit with real data. We focus our attention about the autonomous Models, which easy to handle its existence, uniqueness and stability

Theorem 1.1 (The Fundamental Existence-Uniqueness Theorem). *Let Ω be an open set of $\mathbb{R}^n$ containing x_0 and assume that $f \in C^1(\Omega)$. Then there exists an $a > 0$ such that the initial value problem $\dot{x} = f(x)$, $x(0) = x_0$ has a unique solution $x(t)$ on the interval $t \in [-a, a]$.*

Theorem 1.2 *Let $f(x)$ be a continuous function that satisfy Lipschitz condition on a strip $S = \{(t, x): a \leq t \leq b, -\infty \leq x \leq \infty\}$ that is*

$$|f(x_1) - f(x_2)| \leq K|x_1 - x_2|$$
for some constant $K > 0$ and for all (t, x_1) and (t, x_2) in S . if (t_0, x_0) is an interior point of S , then there exists a unique solution of

$$\dot{x} = f(x), \quad x(0) = x_0 \text{ on the interval } t \in [a, b]$$

1.2 Dynamical systems and Mathematical Modeling

Mathematical modeling and dynamical system theory are essential tools in various scientific and engineering disciplines. From physics to biology, chemistry to economics, mathematical models provided quantitative descriptions of systems, allowing researchers to analyze, predict, and control their behavior. Models were developed to capture a wide range of phenomena, including population dynamics, chemical reactions, ecological systems, and economic trends. The study of dynamical systems and mathematical modeling has a rich historical background that spans several centuries. The origins can be traced back to the development of mathematical techniques for understanding and predicting the behavior of physical systems.

Definition: A dynamical system refers to a mathematical framework used to study the behavior of evolving systems over time. It provides a way to analyze how the state of a system changes as time progresses, based on a set of rules or equations that govern its dynamics.

1.2.1. What is Mathematical Modeling and Modeling Approach

In order to introduce Mathematical Modeling meaning we need first to understand what exactly modeling is?

Modeling is to transform natural observations or laws to a system of equations or a model that can be analyzed with multiple techniques.

We have different kinds of models such as: -

1. Analytical Models
2. Numerical Models
3. Graphical Models

Analytical Models help us to get the exact solutions we are trying to get and the closest to real world data, but sometimes it's hard to formulate analytical models or a system out of differential equations.

Numerical Models use numerical analysis techniques to help us get approximately results close to the exact solutions, as it might be an easy choice for any mathematician yet at sensitive models it's better to get an exact or an analytical solution for the system of differential equations.

Graphical Models is a way of solving differential equations visually using charts or diagrams and helps to simplify complex equations, but it's known for its poor results so usually mathematicians use graphical models to get an overview about the system of equations they are trying to solve or analyze.

Here in our project, we might prefer using numerical models since we will try to solve Lorenz System using Python, but we will consider using small errors to get the results so close to the exact solutions.

In the next few chapters, we will start to explore some important details that will help us to understand how we can study the Model and with which techniques, also we will discuss what's the Jacobian Matrix and it's use for us and how can we find equilibrium points of differential equations and study it's stability and much more.

1.2.2. Equilibrium Points of a system and analysis

Differential equations have equilibrium points, and the number of the equilibrium points can differ from one equation to the another.

Equilibrium points help us to know at which point the rate of change would equal zero.

As an example, let's see what the equilibrium points of the following differential equation is.

$$y' = 5x - 15$$

To find the value of "x" that would make the rate of change equal to zero we must let ($y' = 0$)

$$0 = 5x - 15, \text{ then } x = 3$$

We will discuss in the following chapters how we can study the stability of a differential equation using Jacobian matrix but before we do, is there a way to know if the equilibrium point is stable or not stable?

We could do so, if we tried to evaluate the differential equation with “x” values smaller than 3 we will find out all the answers is negative, and if evaluate with values bigger than 3 the outputs will be positive which means the values diverge from the equilibrium point making it **unstable**.

Finding the equilibrium points is so important in mathematical modeling and analyzing the points leads to many insights as we will see in this project.

1.3. Python Software

Python is a high-level, interpreted, general-purpose programming language. It was created by Guido van Rossum and was first released in 1991. Python is known for its readability, simplicity, and ease of learning, making it one of the most popular languages for both beginners and experienced developers.

Key features of Python•

- **Readable and easy-to-learn syntax**• Python emphasizes code readability, using indentation instead of braces to define blocks.
- **Interpreted language**• Code is executed line by line, which helps with testing and debugging.
- **Versatile**• Python can be used for web development, data analysis, machine learning, automation, scripting, scientific computing, and more.
- **Large standard library**• Python comes with a vast collection of modules and libraries for various tasks.
- **Community support**• Python has a huge, active community with plenty of resources, tutorials, and third-party libraries.

Simple example•

```
print (“hello world!”)
```

popular uses of Python•

- Web development (using frameworks like Django and Flask)
- Data science and machine learning (using libraries like pandas, NumPy, matplotlib, and scikit-learn)
- Automation and scripting
- Game development

1.3.1. Why is python Useful for mathematics?

1. Readability and Simplicity

Python code is often easy to read and write, making it straightforward to express mathematical ideas clearly and concisely.

2. Powerful Mathematical Libraries

Python has a rich ecosystem of libraries for mathematical and scientific computing, including:

- **NumPy**: Fast array operations, linear algebra, and mathematical functions.
- **SciPy**: Advanced mathematics, optimization, and statistics.
- **SymPy**: Symbolic mathematics (like algebra, calculus, and equation solving).
- **Matplotlib and Seaborn**: Graphing, data visualization, and plotting.
- **Pandas**: Data manipulation and analysis, including table-like structures.

3. Supports Symbolic & Numeric Mathematics

With libraries like **SymPy**, Python can handle symbolic mathematics (like manipulating equations algebraically), while **NumPy** and **SciPy** are excellent for numeric computation.

4. Quick Prototyping

Python's interpreted nature allows for quick experimentation and prototyping of mathematical ideas without the need to compile code.

5. Community and Resources

There is a large, active community of mathematicians, scientists, and educators using Python, which means plenty of tutorials, guides, and help are available.

6. Integration with Other Tools

Python works well with other software and file types commonly used in mathematics and science (like Excel, CSV, MATLAB files, etc.).

1.3.2. Why is python useful for mathematical modeling?

1. Rich Ecosystem of Scientific Libraries

- NumPy & SciPy: Allow for advanced numerical computing, linear algebra, optimization, integration, and statistical operations.
- SymPy: Enables symbolic computation, letting you manipulate algebraic expressions and solve equations analytically.
- Pandas: Useful for handling, cleaning, and analyzing large datasets to feed into mathematical models.
- Matplotlib/Seaborn: Provide powerful tools for visualizing model outputs and understanding results.

2. Rapid Prototyping & Ease of Use

- Python's simple, readable syntax allows you to develop and test models quickly.
- Interactive development environments like Jupyter Notebook make experimenting, debugging, and visualizing easy.

3. Flexibility

- Python can handle a wide range of mathematical modeling tasks: discrete or continuous models, deterministic or stochastic systems, simulations, and optimization.

4. Interdisciplinary Applications

- Python is useful across disciplines (engineering, physics, biology, economics, social sciences, etc.), making it easy to collaborate and apply models in diverse areas.

5. Scalability & Integration

- Python can be combined with high-performance libraries (like Numba or Cython) or languages (C/C++/Fortran) for faster computation if needed.
- Easily integrates with other tools for input/output, big data processing, and even machine learning for data-driven modeling.

6. Community & Resources

- A massive, active community publishing resources, tutorials, and open source modeling libraries, ensuring you can find support even for advanced modeling tasks.

Example: mathematical modeling in python (Code)

Solve a logistic growth with carrying capacity $K=1000$, $r=0.2$ and initial population $N=10$ knowing that the differential equation of population growth is

$$\frac{dN}{dt} = rN\left(1 - \frac{N}{K}\right)$$

```
r, K, N0 = 0.2, 1000, 10
t = np.arange(0, 50, 0.1)
N = np.zeros(len(t))
N[0] = N0
for i in range(1, len(t)):
    N[i] = N[i-1] + 0.1 * r * N[i-1] * (1 - N[i-1] / K)
plt.plot(t, N, label='Population')
plt.axhline(K, color='r', linestyle='--', label='K')
plt.xlabel('Time')
plt.ylabel('Population')
plt.title('Logistic Growth')
plt.grid()
plt.legend()
plt.show()
```

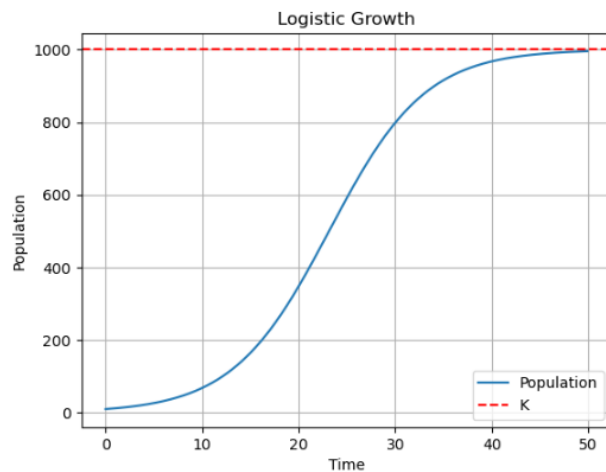


Fig 1.3-1 writing a code visualizing a direct application of mathematical modeling using a model called logistic population growth model.

1.3.3. Python libraries features

What is matplotlib? and uses of it?

Matplotlib is a popular, open-source Python library used for creating static, interactive, and animated visualizations in Python. It provides functions and tools for generating a wide variety of graphs and plots, such as line charts, bar charts, scatter plots, histograms, and more.

Key Features

- **Versatile plotting**• Supports many kinds of plots (line, bar, scatter, histogram, pie, error bars, 3D plots, etc.).
- **Customizable**• Almost every element of a plot (titles, labels, colors, legends, styles) can be customized.
- **Integration**• Works well with scientific libraries like NumPy and pandas, so you can plot directly from data structures.
- **Interactive plots**• Can create plots that allow zooming, panning, and updating in real-time.
- **Export**• Plots can be saved in various formats (PNG, JPG, SVG, PDF).

Common Uses of Matplotlib

- **Data visualization**• Turn data into charts and graphs to make analysis easier and more informative.
- **Exploratory data analysis (EDA)**• Quickly visualize data distributions, trends, and relationships.
- **Scientific and mathematical plotting**• Plot mathematical functions, results from simulations, and experimental data.
- **Reporting and presentations**• Create high-quality figures for documents, reports, and publications.
- **Custom dashboards and GUIs**• Integrate plots into custom data applications.

What is NumPy? and uses of it?

NumPy (Numerical Python) is a fundamental, open-source Python library used for numerical computing. It provides support for creating and manipulating powerful data structures called arrays, as well as functions for performing a wide range of mathematical operations efficiently.

Key Features of NumPy

- **N-dimensional array object (ndarray)**• Handles large, multi-dimensional arrays and matrices of numeric data efficiently.

- **Fast numerical operations**• Vectorized calculations (such as addition, subtraction, multiplication, division) are performed much faster than with standard Python lists.
- **Math functions**• Includes a variety of mathematical, logical, statistical, and Fourier transform functions.
- **Integration**• Works seamlessly with other data science libraries like pandas, SciPy, and matplotlib.

Common Uses of NumPy

1. Numerical Data Storage & Manipulation

- Create and manage powerful arrays and matrices to handle numerical data (1D, 2D, or even higher dimensions).

2. Fast Mathematics and Linear Algebra

- Perform high-speed mathematical operations, including element-wise addition/multiplication, dot products, matrix inversions, solving systems of equations, etc.

3. Data Preparation for Machine Learning

- Many machine learning libraries (like scikit-learn and TensorFlow) expect NumPy arrays as input/output data.

4. Scientific Simulations

- Widely used in physics, engineering, finance, and scientific computing for simulations and complex calculations.

5. Statistical Analysis

- Provides functions for computing mean, median, standard deviation, and other statistical operations on large datasets.

6. Integration with Other Tools

- Easy reading/writing of data from/to text files, CSV, and binary formats.

Example• Basic Array Operations with NumPy (Code)

```
[3]: import numpy as np
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])
print(a + b)
print(a * b)
print(np.mean(a))

[11 22 33 44]
[ 10  40  90 160]
2.5
```

Fig 1.3.2 A code using NumPy library to manipulate matrices like adding them or using multiplication.

What is pandas? and uses of it?

Pandas• is a powerful, open-source Python library designed for data manipulation, analysis, and cleaning. It is especially suited for working with structured (tabular) data, such as spreadsheets, SQL tables, or CSV files.

Key Features of Pandas

- **DataFrame and Series**
 - **DataFrame**• a 2-dimensional, tabular data structure (like a spreadsheet) with labeled rows and columns.
 - **Series**• a 1-dimensional labeled array (like a column in a table).
 - **Data Import/Export**• Read and write data from/to popular formats such as CSV, Excel, SQL databases, JSON, and more.
- **Data Cleaning**• Handle missing data, filter rows/columns, change data types, and preprocess datasets.
- **Data Manipulation**• Sort, filter, group, merge, pivot, and reshape data for analysis.
- **Efficient Operations**• Built on top of NumPy for fast, vectorized operations on large datasets.
- **Time Series Support**• Tools for working with dates, times, and time-indexed data.

Common Uses of Pandas

1. Data Analysis and Exploration

- Inspect, summarize, and understand your data with powerful querying and aggregation tools.

2. Data Cleaning and Preparation

- Remove or fill in missing values, correct data formats, and transform raw data for machine learning or visualization.

3. Handling Large Datasets

- Efficiently manage millions of rows of data that might be unwieldy in Excel or plain Python.

4. Merging and Joining Data

- Combine multiple datasets coming from different sources, much like SQL joins.

5. Time Series Analysis

- Resample, shift, and window data for financial analysis, forecasting, or scientific research.

6. Integration with Other Libraries

- Works smoothly with NumPy, matplotlib, scikit-learn (for machine learning), and more.

What is SciPy? And the uses of it?

SciPy builds on top of NumPy, adding a large collection of complex mathematical algorithms and convenience functions for scientific and engineering tasks. While NumPy handles basic numerical operations and array management, SciPy provides modules for more advanced mathematical functions.

Key Uses of SciPy

1. Numerical Integration

- Calculate definite integrals (area under curves), solve ordinary differential equations (ODEs).

2. Optimization

- Find maxima, minima, and solve constrained or unconstrained optimization problems.

3. Interpolation

- Estimate values between discrete data points using interpolation techniques.

4. Signal Processing

- Operations like filtering, Fourier transforms, and spectral analysis.

5. Linear Algebra

- Advanced matrix operations: decompositions, eigenvalues, solving systems of equations, etc.

6. Statistics

- Probability distributions, hypothesis tests, correlation analysis, and more.

7. File I/O

- Read and write data in specialized scientific file formats (like MATLAB .mat files).

Example: Root Finding with SciPy (Code)

```
[5]: from scipy.optimize import root
def equation(x):
    return x**2 - 9
solution = root(equation, 2)
print(solution.x)

[3.]
```

Fig 1.3.3 using SciPy library to find the value of x of an equation.

Chapter 2

Numerical Methods for ODEs

2.1 Numerical methods vs. analytical methods

Analytical methods involve finding exact solutions to DEs using mathematical techniques such as separation of variables, integrating factors, and Laplace transforms. These methods provide closed-form solutions that can be expressed in terms of known functions, and they offer insights into the behavior of the system being modeled.

Analytical methods are particularly useful for simple DEs, where the equations can be easily solved using algebraic or calculus techniques. However, for complicated (non-linear, chaotic) ODEs, analytical solutions may not exist, or they may be too difficult to find, making numerical methods necessary.

Numerical methods involve approximating the solutions to ODEs using numerical techniques such as Euler's method, Runge-Kutta methods, and finite difference methods. These methods approximate the solution at discrete points in time or space, using a step-by-step process that involves iterating through the DEs using numerical algorithms. Numerical methods are particularly useful for ODEs, where analytical solutions are not available or are difficult to obtain. They can also handle non-linear and discontinuous systems, which are difficult to solve analytically. However, numerical methods have limitations, such as round-off errors, truncation errors, and stability issues, which can affect the accuracy and reliability of the solutions obtained.

2.2 Types of numerical methods

Numerical methods are basic tools for studying models, indeed for non-linear systems which analytical methods became dramatic hard or impossible. In this section we discuss standard finite difference methods for ordinary differential equations, especially for non-linear autonomous systems, here we will show the methods of (Euler's forward & backward methods, Classical Runge Kutta method and Adams method), trying to specify their errors, basic idea, make simple comparison and mention other methods we find.

2.2.1 Euler Method

The standard Euler's method depends on a linear predication of the unknown function by the last point and a step size. we can derive it from the finite difference differentiation approximation

$$y'(x) = \frac{y(x+\Delta x) - y(x)}{\Delta x}$$

Now we can arrange this formula to get an approach to solve the IVP equation

$$y' = f(x, y)$$

$$y(x_0) = y_0$$

to be the following equations refer to explicit Euler's method (Euler Forward)

$$y_{i+1} = y_i + hf(x_i, y_i)$$

$$x_{i+1} = x_i + h$$

Error Analysis

It is obvious that there is a linear approach to solve a differential equation, especially non-linear, there are many errors left behind. Numerical methods for ODEs involve two types of errors:

Truncation error is the consideration of calculate an infinite computation by take up a bound for it, to see what it is look like we take Taylor series expansion about the point (x_i, y_i) .

$$y_{i+1} = y_i + hf(x_i, y_i) + \frac{h^2}{2!} f'(x_i, y_i) + \cdots + O(h^2)$$

$O(h^2)$ is the truncation error of order 2, for Euler's method its order is 2

Round-off error the problem with truncation that is not defined as well we couldn't calculate it so we can take an approximation of it

$$E_a = \frac{h^2}{2!} f''(x, y)$$

2.2.2 Runge Kutta Methods

Runge Kutta methods achieve the accuracy of Taylor series expansion without need to calculate the higher derivatives

$$y_{i+1} = y_i + h\phi(k_1, k_2, k_3, \dots, k_n)$$

where ϕ is called incremental function depending on parameters k_j for $j = 1, 2, \dots, n$. There are many Runge Kutta methods with different orders and different weighted parameters, here we use the 2nd order and the 4th order, then we mention the other we found in references.

2nd Order (Mid-Point)

$$y_{i+1} = y_i + \frac{1}{2}(k_1 + k_2)$$

$$k_1 = hf(x_i, y_i)$$

$$k_2 = hf(x_i + h, y_i + k_1)$$

4th Order (Classical)

$$y_{i+1} = y_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

$$k_1 = hf(x_i, y_i)$$

$$k_2 = hf(x_i + \frac{1}{2}h, y_i + \frac{1}{2}k_1)$$

$$k_3 = hf(x_i + \frac{1}{2}h, y_i + \frac{1}{2}k_2)$$

$$k_4 = hf(x_i + h, y_i + k_3)$$

2.3.3 Adams Methods

Adams method is linear multi-step integration method, which achieves the accuracy dependent on numerical integration techniques as expressed in the following

$$\int_{y_1}^{y_{i+1}} \frac{dy}{dx} dx = \int_{x_i}^{x_{i+1}} f(x, y) dx$$

Which will be reduced in term of y_{i+1} equal to an integral could be integrated by trapezoidal or Simpson method

Adams-Bashforth

Adams Bashforth is an explicit method with β_k has the values according to interpolation coefficient in Table 2.3

$$y_{i+1} = y_i + \sum_{k=0}^n \beta_k f_{i-k}$$

Order	β_0	β_1	β_2	β_3	Local O(h)
1	1				$\frac{1}{2}h^2f'(\xi)$
2	$\frac{3}{2}$	$-\frac{1}{2}$			$\frac{5}{12}h^3f''(\xi)$
3	$\frac{23}{12}$	$-\frac{16}{12}$	$\frac{5}{12}$		$\frac{9}{24}h^4f'''(\xi)$
4	$\frac{55}{24}$	$-\frac{59}{24}$	$\frac{37}{24}$	$-\frac{9}{24}$	$\frac{251}{720}h^5f^{(4)}(\xi)$

Table 2.3: Adams-Bashforth order 1, 2, 3 & 4

2.3.4 Python Software

Example: Solve the initial value problem:

$$y' = -x^3 + 12x^2 - 20x + 8.5$$

for $y(0) = 1$ and $x \in [0, 5]$ with values for step $h = 0.1$,

using Euler method

$$y_{i+1} = y_i + hf(x_i, y_i)$$

We're going to implement the code without using libraries to show the exact procedure of evaluating the function.

```
def function(x, y):
    return -x**3 + 12*x**2 - 20*x + 8.5

def euler_method(function,x0,y0,h,x_end):
    x_values = [x0]
    y_values = [y0]
    while x_values[-1] < x_end:
        x_next = x_values[-1] + h
        y_next = y_values[-1] + h * function(x_values[-1],y_values[-1])
        x_values.append(x_next)
        y_values.append(y_next)
    return np.array(x_values),np.array(y_values)

x_sol , y_sol = euler_method(function, 0 , 1 , 0.1 , 5)

print(y_sol)
```

```
[ 1.      1.85    2.5119  3.0091  3.3644  3.6     3.7375  3.7979
 3.8016  3.7684  3.7175  3.6675  3.6364  3.6416  3.6999  3.8275
 4.04    4.3524  4.7791  5.3339  6.03    6.88    7.8959  9.0891
10.4704 12.05    13.8375 15.8419 18.0716 20.5344 23.2375 26.1875
29.3904 32.8516 36.5759 40.5675 44.83    49.3664 54.1791 59.2699
64.64   70.29    76.2199 82.4291 88.9164 95.68    102.7175 110.0259
117.6016 125.4404 133.5375 141.8875]
```

Fig 2.3-1, We established a function that returns the DE so that we can use it inside the Euler method function, Euler method here is entered inside an iteration until the upper bound in the interval is the point we're on.

Runge Kutta 4th Order

```
[19]: def function(x, y):
        return -x**3 + 12*x**2 - 20*x + 8.5

def Runge_kutta(function,x0,y0,h,x_end):
    x_values = [x0]
    y_values = [y0]
    while x_values[-1] < x_end:
        x_next = x_values[-1] + h

        k1 = h * function(x_values[-1], y_values[-1])
        k2 = h * function(x_values[-1] + (h/2), y_values[-1] + (k1/2))
        k3 = h * function(x_values[-1] + (h/2), y_values[-1] + (k2/2))
        k4 = h * function(x_values[-1] + h, y_values[-1] + k3)
        y_next = y_values[-1] + ((1/6) * (k1 + 2*k2 + 2*k3 + k4))
        x_values.append(x_next)
        y_values.append(y_next)
    return np.array(x_values) , np.array(y_values)

x_sol , y_sol = Runge_kutta(function, 0 ,1 , 0.1 , 5)
print(y_sol)
```

1.	1.753975	2.3316	2.755975	3.0496	3.234375
3.3316	3.361975	3.3456	3.301975	3.25	3.207975
3.1936	3.223975	3.3156	3.484375	3.7456	4.113975
4.6036	5.227975	6.	6.931975	8.0356	9.321975
10.8016	12.484375	14.3796	16.495975	18.8416	21.423975
24.25	27.325975	30.6576	34.249975	38.1076	42.234375
46.6336	51.307975	56.2596	61.489975	67.	72.789975
78.8596	85.207975	91.8336	98.734375	105.9076	113.349975
121.0576	129.025975	137.25	145.723975]		

Fig 2.3-2 Applying the same problem mentioned in this section using RK4

Chapter 3

Lorenz system

The Lorenz system is a system of ordinary differential equations first studied by mathematician and meteorologist Edward Lorenz.

Lorenz System is combined of three ODE related non-linear equations, our model behaviors are known for having notable chaotic solutions at specific parameters and initial conditions “Butterfly effect”.

That shows us that chaotic actions are deterministic yet it's impossible to predict over a long period of time.

Lorenz system's chaotic solutions make it easy for mathematicians to use the model in many applications such as

- Finance and Economics.
- Climate Modeling.
- Artificial Intelligence (sometimes big neural networks can have chaotic attractors that are being studied by our system).

Model Equations•

$$X' = \alpha(y - x)$$

$$Y' = x(\rho - z) - y$$

$$Z' = xy - \beta z$$

Where X' , Y' and Z' are the system's variables, and α , ρ , and β are parameters

3.1 Finding equilibrium points

$$\text{let } X', Y', Z' = 0 \alpha(y - x) = 0 \text{ Then } y = x \ Y' = 0 \rightarrow x(\rho - z) - y = 0$$

$$\text{substitute } y = x$$

$$x(\rho - z) - x \rightarrow x(\rho - z - 1) \text{ then } x = 0 \text{ or } z = \rho - 1$$

$$Z' = 0, \ xy - \beta z = 0, \text{ substitute } x = y, \rightarrow x^2 - \beta(\rho - 1) = 0$$

then $x = \pm\sqrt{\beta(\rho - 1)}$ Which indicates having three equilibrium points

$$(0,0,0), (\sqrt{\beta(\rho - 1)}, \sqrt{\beta(\rho - 1)}, \rho - 1), (-\sqrt{\beta(\rho - 1)}, -\sqrt{\beta(\rho - 1)}, \rho - 1)$$

Now we see that the model will have three fixed points IFF $\rho > 1$ since the Square root will give an imaginary number if ρ is smaller than or equal to 1 which will lead us to have only the origin Point

3.2 Solving the System of Differential Equations using Python

We will use the libraries NumPy, SciPy and Matplotlib. First, we define our system and the parameters knowing that the system will be chaotic at these exact parameters, Second, we define the time interval, and the method used to solve the differential equations, then we're going to visualize the trajectory of Lorenz model.

Lorenz Model is:

$$X' = \alpha(y - x)$$

$$Y' = x(\rho - z) - y$$

$$Z' = xy - \beta z$$

And we're going to use the Runge-Kutta method; it's a method that has a high accuracy and acceptable error $O(h^4)$.

We will use time-step = 0.001 until the value of "t" in the integration reaches 200.

```
[67]: from scipy.integrate import solve_ivp

def Lorenz(t, state, sigma=10, beta=8/3, rho=28):
    x, y, z = state
    dx_dt = sigma * (y - x)
    dy_dt = x * (rho - z) - y
    dz_dt = x * y - beta * z
    return [dx_dt, dy_dt, dz_dt]

t_span = (0, 200)
t_eval = np.arange(0, 200.001, 0.001)
initial_state = [1.0, 1.0, 1.0]
sol = solve_ivp(fun = Lorenz, t_span = t_span, y0= initial_state, method = 'RK45', t_eval= t_eval)

[69]: t = sol.t
      x, y, z = sol.y
```

Fig 3.2-1, Solving the Lorenz System with a time span With RK-modified

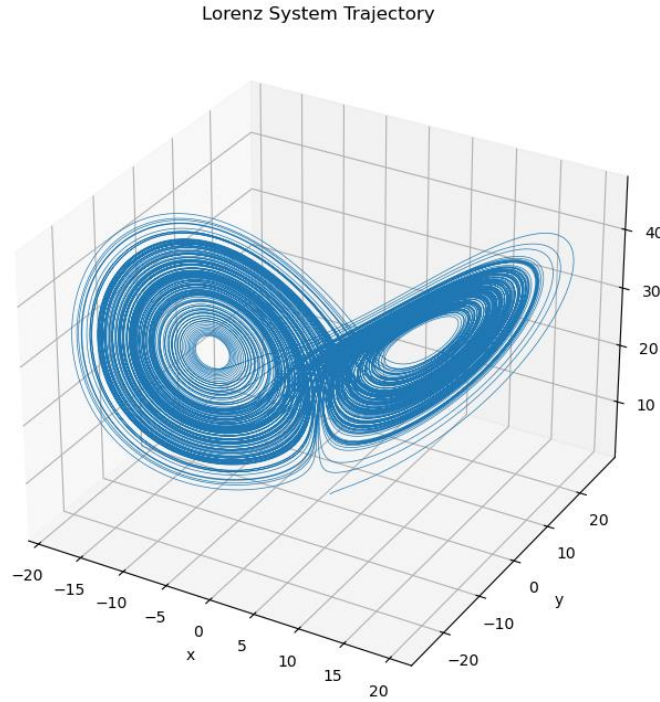


Fig 3-2, Showing the results of solving the Lorenz System Using RK-modified while using chaotic parameters.

3.3 Stability Analysis:

What is Jacobian matrix: Jacobian matrix is a matrix of partial derivatives. Jacobian is the determinant of the Jacobian matrix. The matrix will contain all partial derivatives of a vector function. The main use of Jacobian is found in the transformation of coordinates

Suppose $f: \mathbf{R}^n \rightarrow \mathbf{R}^m$ is a function such that each of its first-order partial derivatives exists on $\mathbf{R}^n$. This function takes a point $x \in \mathbf{R}^n$ as input and produces the vector $f(x) \in \mathbf{R}^m$ as output. Then the Jacobian matrix of " f ", denoted $J_f \in \mathbf{R}^{m \times n}$, is defined such that its $(i, j)^{\text{th}}$ entry is $\frac{\partial F_i}{\partial x_j}$, or explicitly

$$J = \begin{bmatrix} \frac{\partial F_1}{\partial x_1} & \dots & \frac{\partial F_1}{\partial x_n} \\ \frac{\partial F_m}{\partial x_1} & \dots & \frac{\partial F_m}{\partial x_n} \end{bmatrix} \text{ where } J \text{ is the Jacobian matrix}$$

If the values of the eigenvalues is positive this means that the system is **unstable** and if the values is negative the system will be **Asymptotically Stable**.

Here are the conditions for the equilibrium points stability: -

Eigenvalues	Type of Critical Point	Stability
$\lambda_1 > \lambda_2 > 0$	Node	Unstable
$\lambda_1 < \lambda_2 < 0$	Node	Asymptotically Stable
$\lambda_2 < 0 < \lambda_1$	Saddle Point	Unstable
$\lambda_1 = \lambda_2 > 0$	Proper or Improper node	Unstable
$\lambda_1 = \lambda_2 < 0$	Proper or Improper node	Asymptotically Stable
$\lambda_1, \lambda_2 = \mu \pm i\nu$		
$\mu > 0$	Spiral Point	Unstable
$\mu < 0$	Spiral Point	Asymptotically Stable
$\mu = 0$	Center	Stable

Table 3.3.1 Classification of eigenvalues and stability

To calculate the stability of the equilibrium points of Lorenz system we will use Jacobian matrix to get the values of λ ("Eigenvalues") and see if it's Asymptotically stable or Unstable.

We're going to use the following parameters: $\alpha = 10$, $\beta = 8/3$, $\rho = 28$

After substituting the parameters values, we will have the system equations as follows:

$$\begin{aligned} X' &= 10(y - x), \\ Y' &= x(28 - z) - y \\ Z' &= xy - \frac{8}{3}z \end{aligned}$$

After calculating the Jacobian matrix, the matrix will look like this

$$J = \begin{bmatrix} -\alpha - \lambda & 10 & 0 \\ \rho - z & -1 - \lambda & -x \\ y & x & -\beta - \lambda \end{bmatrix}$$

which will equal to

$$J = \begin{bmatrix} -10 - \lambda & 10 & 0 \\ 28 - z & -1 - \lambda & -x \\ y & x & -\frac{8}{3} - \lambda \end{bmatrix}$$

- After evaluating and getting the values of λ for the point (0,0,0) we get these 3 values
- $\lambda_1 = -8/3$, $\lambda_2 = -22.8$ and $\lambda_3 = 11.8$
- And since there is a λ that is positive the point will be **Unstable**.
- As it appears in the photo, which is a graph of the Lorenz system flow, the red x point is the origin point and it's obvious that the point having all the trajectories diverging away from the point.

Checking the two other equilibrium points:

After getting the eigen values of the two other points by the same Jacobian matrix numerically or getting an exact solution we will find out that they equal to $\lambda_1 = -13.85$

$$\lambda_2, \lambda_3 = 0.094 \pm 10.19i$$

Which means that those two points are also **Unstable**.

As we discussed earlier, Lorenz system with specific parameters can have chaotic solutions which is obvious from the shape of the trajectories shown in the last slide, in our analysis we chose parameters that will get a chaotic solution since it would be better to study such a case but is there a way to find a relation to know if our parameters is going to get chaotic solutions or not? Mathematicians have found out that the next relation defines it.

We can only have stable equilibrium points if $\rho > \alpha \frac{2(\alpha+\beta+1)}{\alpha+1+\sqrt{(\alpha-1)^2+4\alpha\rho}}$

Chapter 4

Machine Learning Applications on Dynamical Systems

4.1 Introduction to Machine Learning

Machine Learning Definition: An algorithm developed in the field of Artificial Intelligence to help the machine make decisions built on statistical approaches without explicit orders.

Machine Learning Types

Supervised Learning: Machine Learning algorithms that learn from labeled data to predict unknown values.

Unsupervised Learning: Machine Learning algorithms that learn from unlabeled data without explicit guidance.

Reinforcement Learning: Machine Learning algorithms that develop the machine to make decisions built on a reward signal.

Classification: A supervised algorithm that is used on categorical data labels and predicts the category of the unlabeled data.

How does Classification Algorithm work?

We try to find a relation between the features or variables provided and the outcome we aim to predict.

We use a function called Sigmoid function that has a threshold value at 0.5, sigmoid function aim to transform all values of the outcome in the training dataset into a probability value between [0,1].

Sigmoid Function: $\alpha(z) = \frac{1}{1+e^{-z}}$

“z” value is linear combination between the features of the dataset.

Every feature value in the linear combination has a coefficient that represents the weight of the features known as the importance of a specific feature.

After evaluating the outputs from the sigmoid function, we use the log-likelihood to find the error of the coefficients we used in the linear combination

Log-Likelihood

$$\sum_{i=1}^m [y_i \log(\alpha(x_i^T \beta)) + (1 - y_i) \log(1 - \alpha(x_i^T \beta))]$$

Using Gradient descent and iterative update of coefficient values

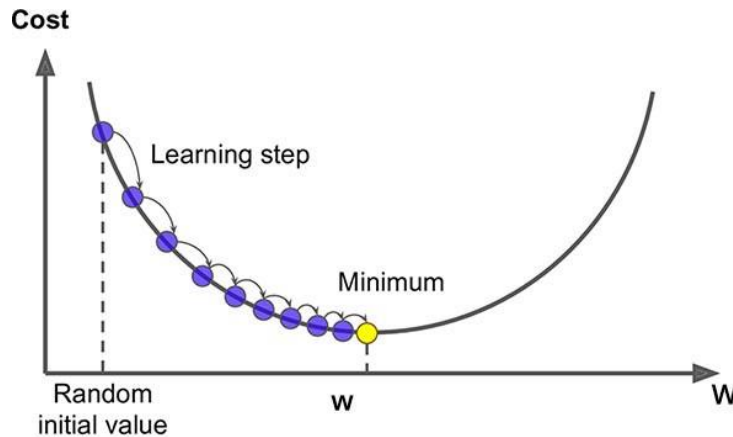


Fig 4.1-1 visualization of gradient descent presenting how iteration led to minimum cost

Gradient Descent is a method used to minimize the error we get from applying statistical models, since in statistical models we have at first to guess the value which is presented in the figure as the “Random initial value”. Of course randomly choosing a value wouldn’t correctly be the nearest value we target from the exact solution if it exists. So, we keep updating the value with a small step size by using gradient descent and the value from the log-likelihood to reach the minimum cost we can have.

After finding the linear combination coefficients that lead to the highest accuracy of the model the iterations stop.

In our case study we will use a model in the library Scikit-Learn that’s called Random Forest Classifier.

Random Forest: Random forest is decision trees that can have either different coefficient values or using different classification approaches, then ensemble the solutions to get more accuracy.

Random forest randomness and the insurance of divergence between the trees makes sure the trees are not dependent.

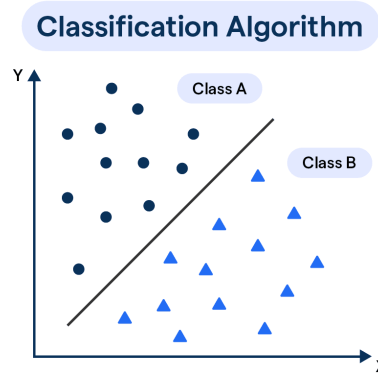


Fig 4.1-2 Explaining how classification algorithms work in the data points plane.

4.2 Preprocessing the data

To apply classification, we need first to preprocess the data to change the categories of the evaluation feature from alphabetical to binary outputs.

Our data consists of 3 features $[\alpha, \beta, \rho]$ which determine and control the state of our system whether it will produce Chaotic solutions or non-chaotic solutions.

The data sigma, Beta values are fixed since the implicit inequality has 3 values we let the rho value to be the factor that changes the state of the system.

The data consists of 138 record of parameters that lead to chaotic solutions, and 62 that lead to non-chaotic solutions.

	sigma	beta	rho	evaluation
0	10	2.666667	13.848846	Chaotic
1	10	2.666667	6.386327	Non-Chaotic
2	10	2.666667	7.153270	Non-Chaotic
3	10	2.666667	11.097237	Chaotic
4	10	2.666667	10.564823	Chaotic
...	...	...	...	...
195	10	2.666667	5.889989	Non-Chaotic
196	10	2.666667	12.940941	Chaotic
197	10	2.666667	2.352281	Non-Chaotic
198	10	2.666667	9.282064	Chaotic
199	10	2.666667	27.673122	Chaotic

200 rows × 4 columns

Fig 4.2-1 Representing the tabular form of the data which shows 3 features and 1 output in 200 records.

We're going to choose a preprocessing method in Scikit-Learn library called One Hot Encoder which transforms categorical alphabetical values into numerical categories.

We state 0 to represent the output “non-chaotic”,

And the state 1 to represent the output “Chaotic”.

```
from sklearn.preprocessing import OneHotEncoder
import pandas as pd

encoder = OneHotEncoder(drop='first', sparse_output=False, categories=[['Non-Chaotic', 'Chaotic']])

encoded_data = encoder.fit_transform(df[['evaluation']])

df_encoded = pd.DataFrame(encoded_data, columns=['evaluation_edited'])

df_modified = pd.concat([df.drop(columns=['evaluation']), df_encoded], axis=1)
```

Fig 4.2-2 Python code that applies an encoder method to transform categories into numerical categories.

	sigma	beta	rho	evaluation_edited
0	10	2.666667	13.848846	1.0
1	10	2.666667	6.386327	0.0
2	10	2.666667	7.153270	0.0
3	10	2.666667	11.097237	1.0
4	10	2.666667	10.564823	1.0
...	...	...	...	...
195	10	2.666667	5.889989	0.0
196	10	2.666667	12.940941	1.0
197	10	2.666667	2.352281	0.0
198	10	2.666667	9.282064	1.0
199	10	2.666667	27.673122	1.0

200 rows × 4 columns

Fig 4.2-3The output of applying the One Hot Encoder on the dataset.

Next step is that we split our dataset into training and testing data with a chosen percentage of the test size.

Splitting the data would be applied on two variables x, y.

X should be the dataframe containing the features without the output column.

Y should be the dataframe containing only the output column.

Then we apply the Random Forest Classifier on the training data to train the model to find the relation and classify our system.

```
x = df_modified[['sigma', 'beta', 'rho']]
y = df_modified['evaluation_edited']
x_train , x_test, y_train, y_test = train_test_split(x,y,test_size=0.25, random_state=42)
classifier = RandomForestClassifier(n_estimators=100, random_state=42)
classifier.fit(x_train, y_train)
```

Fig 4.2-4 Showing the code of applying the random forest classifier on the training set of our data.

4.3 Results and analysis

After running the model, we find out the accuracy of the model is 100% which in most cases is considered as overfitting which means that the classifier trained perfectly on the training data but performs poorly on unknown labeled data.

```
[75]: classifier.score(x_test,y_test)

[75]: 1.0
```

Fig 4.3-1 running the accuracy function to see how well the model will get if we apply it to unknown values.

But in our case, this is a perfect normal output of our application, Why?

Well since Classification models was intentionally built to be applied to real world applications which is imperfect and provide missy data, Additionally in the real world applications we can't find exactly all the features that has relation between our output but in this case we are running our application on an abstract system of differential equations so we know exactly what features affect our model outputs.

The Classifier in the iterations easily found out the relation between the ρ value and α, β that leads the model to either chaotic or non-chaotic solutions.

Which means statistical Learning can be applied in the Mathematical Modeling field for the Analysis of dynamical Systems if it's difficult or impossible to find out the relation between the parameters that lead to specific outputs.

References

- [1] Gareth James, An Introduction to Statistical Learning,
- [2] Robert L. Borelli, Differential Equations A modeling perspective.
- [3] Dr. Salma A. Shatta, Mathematical Modeling Course.